

기존에 작성한 벌금 관리 프로그램에 이중 리스트, 이중 순회(표 출력), 파일 저장(txt), 예외 처리(try-except) 기능을 도입해 고도화하려고 해. 이 4가지 조건이 효율적으로 반영될 수 있도록 전체적인 코드 개선 방향과 논리 구조를 가이드해 줘



아주 훌륭한 접근입니다! 단순히 코드를 이어 붙이는 수준을 넘어, 데이터를 체계적으로 관리하고 프로그램의 안정성까지 고려하는 완벽한 리팩토링(Refactoring) 계획이네요.

요청하신 4가지 핵심 기능이 전체 코드에 매끄럽게 녹아들 수 있도록, 각 기능의 설계 방향과 논리 구조를 가이드해 드릴게요.

1. 이중 리스트 (2D List) 도입: 데이터 구조화

기존에는 벌금만 모아두는 1차원 리스트(`fees`)를 사용했다면, 이제는 조원 한 명의 상세 정보를 한 덩어리로 묶어서 관리해야 합니다.

- 구조 설계: `member_data` 라는 빈 리스트를 만들고, 데이터가 입력될 때마다 `['이름', '상태(문자열)', '지각시간(정수)', '벌금(정수)']` 형태의 리스트를 통째로 `append` 하세요.
- 로직 최적화 팁: 기존 코드에서는 0분 지각(정상 출석)과 실제 지각을 `if-else` 로 나누어 처리했었죠? 이제는 정상 출석한 사람의 데이터(예: `['홍길동', '정상출석', 0, 0]`)도 표에 출력해야 하므로 분기점을 하나로 통합하는 것이 좋습니다. 조건문 중복을 줄이고 일관되게 데이터를 누적해 보세요.

2. 이중 순회 (Nested Loops): 깔끔한 표 출력

`member_data` 에 쌓인 2차원 데이터를 꺼내어 사용자에게 보기 좋게 보여줄 차례입니다.

- 논리 구조: 바깥쪽 `for` 문으로 사람(row)을 한 명씩 꺼내고, 안쪽 `for` 문으로 그 사람의 상세 정보(item)를 하나씩 꺼내어 가로로 출력합니다.
- 출력 포매팅 가이드: 파이썬의 f-string 정렬 기능인 `^` 를 활용해 보세요. `print(f"{str(item):^10}", end=" | ")` 형식으로 작성하면, 데이터의 길이가 달라도 항상 10칸의 공간 안에서 가운데 정렬이 되어 아주 반듯한 표(Table) 모양을 만들 수 있습니다. (출력이 끝나면 다음 사람을 위해 빈 `print()` 로 줄바꿈을 해주는 것도 잊지 마세요!)

3. 예외 처리 (try-except): 튕김 방지 시스템

사용자는 항상 우리가 의도한 대로 값을 입력하지 않습니다. 숫자 입력란에 문자를 넣어도 프로그램이 죽지 않도록 방어막을 쳐야 합니다.

- 적용 포인트: `int(input())` 으로 지각 시간이나 메뉴 번호를 입력받는 모든 곳이 타겟입니다.
- 논리 구조: `while True:` 무한 루프 안에 `try-except ValueError:` 구문을 넣으세요. 정상적인 숫자가 입력되면 `break` 로 루프를 탈출하고, 문자가 입력되어 에러가 발생하면 `except` 블록에서 경고 메시지를 띄운 뒤 다시 입력을 요구하도록 논리를 구성하면 완벽합니다.

4. 파일 저장 (File I/O): 영구적인 정산 기록

프로그램이 종료되어도 정산 내역이 날아가지 않도록 텍스트 파일(txt)로 내보내는 기능입니다.

- 안전한 설계: 파이썬의 `with open(파일명, 'w', encoding='utf-8') as f:` 구문을 사용하세요. 이 방식을 쓰면 파일 쓰기가 끝난 후 별도로 `f.close()` 를 하지 않아도 파이썬이 알아서 파일을 안전하게 닫아줍니다.
- 로직 적용: 메인 메뉴에서 '저장' 옵션을 선택하거나, '종료'를 누르기 직전에 이 파일 저장 함수가 자동으로 실행되도록 배치해 보세요. 파일 작성 시에도 `for` 문을 이용해 `member_data` 안의 요소들을 원하는 문장 형태로 조합해 `f.write()` 로 적어주면 됩니다.

💡 다음 스텝 제안

이렇게 구조를 잡고 나니 코드를 어떻게 뜯어고쳐야 할지 큰 그림이 그려지시나요? 위 가이드를 바탕으로 직접 뼈대를 세워보시고, 막히는 부분이나 에러가 발생하는 구문이 있다면 언제든지 편하게 그 부분만 떼어와서 질문해 주세요!